

PRAKTIKUM WORKSHOP KOMUNIKASI MULTIMEDIA
LAPORAN PROJECT CLIENT DAN SERVER
SMILE DETECTOR



Dosen pengampu :

Ginanjjar Suwasono Adi, SST, MSc

Oleh :

Febi Mauna Ari Pritama	2231130090
Joela Cataluna Virginia Fouk	2231130028
Tsania A'yunina	2231130030

PROGRAM STUDI TELEKOMUNIKASI
JURUSAN TEKNIK ELEKTRO
POLITEKNIK NEGERI MALANG

2024

Daftar Isi

Smile Detector.....	3
Server	5
Client	10
Hasil Praktikum	13

Smile Detector

Smile detector adalah teknologi pengenalan wajah yang secara khusus dirancang untuk mendeteksi dan menganalisis senyuman dalam gambar atau video. Teknologi ini banyak digunakan dalam berbagai aplikasi, termasuk fotografi, desain antarmuka pengguna, dan bahkan dalam studi psikologis. Komponen Utama dan Fungsionalitas:

- **Akuisisi Gambar:** Langkah pertama melibatkan pengambilan gambar atau frame video menggunakan kamera. Kamera dengan resolusi tinggi memastikan akurasi yang lebih baik dalam mendeteksi fitur wajah.
- **Deteksi Wajah:** Sistem mengidentifikasi keberadaan wajah dalam gambar. Ini biasanya dilakukan menggunakan algoritma seperti Haar cascades, Histogram of Oriented Gradients (HOG), atau pendekatan berbasis deep learning seperti Convolutional Neural Networks (CNNs).
- **Deteksi Landmark Wajah:** Setelah wajah terdeteksi, sistem mengidentifikasi titik-titik kunci pada wajah. Titik-titik ini termasuk area di sekitar mata, hidung, mulut, dan garis rahang. Alat seperti Dlib's facial landmark detector atau Mediapipe's face mesh dapat digunakan untuk tujuan ini.
- **Pengenalan Senyuman:** Sistem menganalisis area di sekitar mulut untuk mendeteksi senyuman. Ini melibatkan pengukuran kelengkungan bibir, keterbukaan mulut, dan faktor-faktor lain yang menunjukkan senyuman. Detektor senyuman yang lebih canggih menggunakan model pembelajaran mesin yang dilatih pada dataset besar yang berisi wajah yang tersenyum dan tidak tersenyum.
- **Klasifikasi:** Sistem mengklasifikasikan ekspresi wajah yang terdeteksi sebagai senyuman atau tidak. Ini biasanya dilakukan menggunakan classifier seperti Support Vector Machines (SVM), decision trees, atau neural networks.
- **Output:** Output akhir menunjukkan apakah senyuman terdeteksi atau tidak. Ini bisa berupa output biner (senyum atau tidak) atau skor probabilitas yang menunjukkan kemungkinan adanya senyuman.

Aplikasi:

- **Fotografi:** Dalam kamera dan smartphone, deteksi senyuman digunakan untuk secara otomatis mengambil foto saat orang tersenyum.
- **Pengalaman Pengguna:** Meningkatkan antarmuka pengguna dengan merespons emosi pengguna, seperti tersenyum untuk mengonfirmasi suatu tindakan.
- **Pemasaran:** Menganalisis reaksi pelanggan terhadap produk atau iklan.
- **Kesehatan:** Memantau kesejahteraan emosional pasien melalui ekspresi wajah mereka.

Teknologi dan Algoritma:

- **Haar Cascade Classifiers:** Metode awal untuk deteksi wajah, masih digunakan dalam beberapa detektor senyuman untuk deteksi wajah awal.

- Deep Learning: Convolutional Neural Networks (CNNs) secara luas digunakan untuk deteksi wajah dan pengenalan senyuman karena akurasi yang tinggi dan kemampuan untuk mempelajari pola yang kompleks.
- OpenCV: Pustaka visi komputer sumber terbuka yang menyediakan alat untuk pemrosesan gambar, deteksi wajah, dan deteksi landmark wajah.
- Dlib: Toolkit yang berisi algoritma pembelajaran mesin dan alat untuk membuat perangkat lunak yang kompleks, termasuk deteksi landmark wajah.
- Mediapipe: Kerangka kerja yang dikembangkan oleh Google untuk membangun pipeline pembelajaran mesin multimodal, termasuk deteksi wajah dan deteksi landmark wajah.

Server

```
import socket
import cv2
import numpy as np
import pickle
import struct
```

- socket: Untuk membuat koneksi jaringan.
- cv2: OpenCV untuk menangkap dan memproses video.
- pickle: Untuk serialisasi data (mengubah objek menjadi byte stream).
- struct: Untuk mengemas data dalam format biner.

```
faceCascade = cv2.CascadeClassifier('d:/KULIAH/MULTIMEDIA/smile_detection-
master/haarcascade_frontalface_default.xml')
smileCascade = cv2.CascadeClassifier('d:/KULIAH/MULTIMEDIA/smile_detection-
master/haarcascade_smile.xml')
```

□ **cv2.CascadeClassifier:**

- Fungsi ini digunakan untuk memuat model Haar Cascade yang telah dilatih. Model ini digunakan untuk mendeteksi objek dalam gambar atau video, seperti wajah, mata, senyuman, dll.
- Haar Cascade adalah algoritma yang digunakan untuk pendeteksian objek berbasis metode cascade, yang terdiri dari serangkaian tes sederhana yang diterapkan pada gambar untuk mendeteksi fitur tertentu.

□ **faceCascade:**

- Baris kode ini memuat model Haar Cascade yang digunakan untuk mendeteksi wajah manusia.
- 'd:/KULIAH/MULTIMEDIA/smile_detection-master/haarcascade_frontalface_default.xml' adalah path ke file XML yang berisi model Haar Cascade yang dilatih untuk deteksi wajah depan.
- faceCascade adalah objek CascadeClassifier yang digunakan untuk mendeteksi wajah dalam gambar atau video.

□ **smileCascade:**

- Baris kode ini memuat model Haar Cascade yang digunakan untuk mendeteksi senyuman.
- 'd:/KULIAH/MULTIMEDIA/smile_detection-master/haarcascade_smile.xml' adalah path ke file XML yang berisi model Haar Cascade yang dilatih untuk deteksi senyuman.
- smileCascade adalah objek CascadeClassifier yang digunakan untuk mendeteksi senyuman dalam gambar atau video.

```

• # Create socket
• server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
• server_socket.bind(('192.168.16.93', 8000))
• server_socket.listen(1)
• print("Server listening on port 8000")

```

□ Membuat Socket:

- `server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)`
 - `socket.AF_INET`: Menentukan bahwa kita akan menggunakan alamat IPv4.
 - `socket.SOCK_STREAM`: Menentukan bahwa kita akan menggunakan protokol TCP, yang merupakan protokol berbasis koneksi yang andal.

□ Mengikat Socket ke Alamat IP dan Port:

- `server_socket.bind(('192.168.16.93', 8000))`
 - `bind()`: Metode ini mengikat socket ke alamat IP dan port yang ditentukan. Dalam hal ini, IP address 192.168.16.93 dan port 8000.
 - '192.168.16.93': IP address dari server. Ini adalah alamat di mana server akan mendengarkan koneksi masuk.
 - 8000: Port yang akan digunakan untuk mendengarkan koneksi.

□ Menyiapkan Socket untuk Menerima Koneksi Masuk:

- `server_socket.listen(1)`
 - `listen()`: Metode ini menempatkan socket dalam mode mendengarkan.
 - 1: Parameter ini menentukan jumlah maksimal koneksi yang bisa menunggu dalam antrian. Dalam hal ini, server akan menunggu hingga satu koneksi masuk.

□ Menampilkan Pesan bahwa Server Sedang Mendengarkan:

- `print("Server listening on port 8000")`
 - Mencetak pesan ke konsol yang menunjukkan bahwa server sedang mendengarkan koneksi masuk pada port 8000.

```

• conn, addr = server_socket.accept()
• print(f"Connection from {addr}")

```

□ Menerima Koneksi Masuk:

- `conn, addr = server_socket.accept()`
 - `accept()`: Metode ini digunakan untuk menerima koneksi masuk dari klien. Metode ini memblokir (menunggu) hingga ada koneksi yang diterima.
 - Ketika koneksi diterima, `accept()` mengembalikan dua nilai:
 - `conn`: Sebuah objek socket baru yang digunakan untuk berkomunikasi dengan klien.
 - `addr`: Alamat dari klien yang terhubung. Ini adalah tuple yang terdiri dari alamat IP dan nomor port klien.
 -

□ Menampilkan Alamat Klien yang Terhubung:

- `print(f'Connection from {addr}')`
 - F-string (formatted string) digunakan untuk menyisipkan nilai `addr` langsung ke dalam string yang dicetak.
 - Menampilkan pesan ke konsol yang menunjukkan alamat IP dan port dari klien yang terhubung.

```
• while True:
•     data = b""
•     payload_size = struct.calcsize("Q")
•     while len(data) < payload_size:
•         packet = conn.recv(1024) # was 256*1024
•         if not packet: break
•         data += packet
•     packed_msg_size = data[:payload_size]
•     data = data[payload_size:]
•     msg_size = struct.unpack("Q", packed_msg_size)[0]
•
•     while len(data) < msg_size:
•         data += conn.recv(1024) # was 256*1024
•     frame_data = data[:msg_size]
•     data = data[msg_size:]
•
•     frame = pickle.loads(frame_data)
•     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
•     faces = faceCascade.detectMultiScale(
•         gray,
•         scaleFactor=1.3,
•         minNeighbors=5,
•         minSize=(30, 30)
•     )
•
•     for (x, y, w, h) in faces:
•         cv2.rectangle(frame, (x, y), (x+w, y+h), (255, 0, 0), 2)
•         roi_gray = gray[y:y+h, x:x+w]
•
•         smile = smileCascade.detectMultiScale(
•             roi_gray,
•             scaleFactor=1.5,
•             minNeighbors=15,
•             minSize=(25, 25),
•         )
•
•         for i in smile:
•             if len(smile) > 1:
•                 cv2.putText(frame, "Smile", (x, y-30), cv2.FONT_HERSHEY_SIMPLEX,
•                     2, (0, 255, 0), 3, cv2.LINE_AA)
•
•     result_data = pickle.dumps(frame)
```

```

• message_size = struct.pack("Q", len(result_data))
• conn.sendall(message_size + result_data)
•
• conn.close()

```

❑ **Menerima Data dari Klien:**

- Kode ini menerima data dari klien dalam loop yang berjalan terus menerus (while True), sampai klien mengakhiri koneksi.

❑ **Menangani Ukuran Payload:**

- payload_size = struct.calcsize("Q"): Mendapatkan ukuran payload yang akan diterima. Q adalah format untuk unsigned long long (8 bytes).
- while len(data) < payload_size: Loop untuk menerima data hingga ukuran payload terpenuhi.
- packet = conn.recv(1024): Menerima paket data sebesar 1024 bytes dari klien.
- data += packet: Menambahkan data yang diterima ke buffer data.

❑ **Memproses Ukuran Pesan:**

- packed_msg_size = data[:payload_size]: Mengambil bagian pertama dari data yang berisi ukuran pesan.
- data = data[payload_size:]: Memotong ukuran pesan dari data yang diterima.
- msg_size = struct.unpack("Q", packed_msg_size)[0]: Mengekstrak ukuran pesan sebenarnya dari data yang diterima.

❑ **Menerima Data Frame:**

- while len(data) < msg_size: Loop untuk menerima data hingga ukuran pesan terpenuhi.
- data += conn.recv(1024): Menerima sisa data dari klien.
- frame_data = data[msg_size:]: Mengambil data frame dari buffer.
- data = data[msg_size:]: Memotong data frame dari buffer.

❑ **Menggunakan Pickle untuk Deserialisasi Data Frame:**

- frame = pickle.loads(frame_data): Mendekode data frame yang diterima menjadi objek gambar.

❑ **Mengonversi Frame ke Grayscale:**

- gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY): Mengonversi frame ke gambar grayscale untuk deteksi wajah dan senyuman.

❑ **Deteksi Wajah Menggunakan Haar Cascade:**

- faces = faceCascade.detectMultiScale(...): Menggunakan model Haar Cascade untuk mendeteksi wajah dalam gambar grayscale.

□ **Menggambar Kotak di Sekitar Wajah dan Deteksi Senyuman:**

- `cv2.rectangle(frame, (x, y), (x+w, y+h), (255, 0, 0), 2)`: Menggambar kotak di sekitar wajah yang terdeteksi.
- `roi_gray = gray[y:y+h, x:x+w]`: Mendefinisikan area wajah dalam gambar grayscale.
- `smile = smileCascade.detectMultiScale(...)`: Menggunakan model Haar Cascade untuk mendeteksi senyuman dalam area wajah.
- `if len(smile) > 1`: Jika ada lebih dari satu senyuman yang terdeteksi.
- `cv2.putText(frame, "Smile", (x, y-30), cv2.FONT_HERSHEY_SIMPLEX, 2, (0, 255, 0), 3, cv2.LINE_AA)`: Menambahkan teks "Smile" di atas kotak wajah jika senyuman terdeteksi.

□ **Mengirim Kembali Frame yang Diproses ke Klien:**

- `result_data = pickle.dumps(frame)`: Men-serialisasi frame yang diproses menjadi byte stream menggunakan pickle.
- `message_size = struct.pack("Q", len(result_data))`: Mengemas ukuran pesan menggunakan struct.
- `conn.sendall(message_size + result_data)`: Mengirim ukuran pesan dan data frame ke klien.

□ **Menutup Koneksi:**

- `conn.close()`: Menutup koneksi setelah loop utama selesai.

Client

```
import socket
import cv2
import pickle
import struct
def run_server():
```

- socket: Digunakan untuk membuat koneksi jaringan antara client dan server.
- cv2: OpenCV digunakan untuk mengambil video dari webcam atau file video.
- pickle: Digunakan untuk mengonversi objek Python menjadi byte stream.
- struct: Digunakan untuk mengatur format data yang akan dikirimkan. Membuat objek socket menggunakan fungsi socket.socket().

```
def receive_data(sock, count):
    buf = b''
    while count:
        newbuf = sock.recv(count)
        if not newbuf: return None
        buf += newbuf
        count -= len(newbuf)
    return buf
```

- Fungsi receive_data mengambil dua argumen: sock (socket) yang merupakan koneksi socket dan count yang menentukan jumlah byte yang akan diterima. Fungsi ini membaca data dari socket sock sebanyak count byte, dan mengembalikan data yang berhasil dibaca. Jika tidak ada data yang diterima (karena koneksi terputus), fungsi mengembalikan None. Mengikat socket server dengan ke alamat ip dan port

```
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(('192.168.16.93', 8000))
cap = cv2.VideoCapture(0)
```

- socket.socket(socket.AF_INET, socket.SOCK_STREAM): Membuat objek socket untuk koneksi TCP/IP dengan menggunakan alamat IPv4
- client_socket.connect(('192.168.16.93', 8000)): Membuat koneksi ke server dengan alamat IP '192.168.16.93' dan port 8000.
- cv2.VideoCapture(0): Membuka kamera pada perangkat dengan indeks 0. Ini menginisialisasi objek cap yang digunakan untuk membaca frame dari kamera. Siapkan socket untuk menerima koneksi dengan mengikatnya ke alamat IP dan port yang telah kita tentukan sebelumnya.

```

while True:
    ret, frame = cap.read()
    frame = cv2.resize(frame, (640, 480)) # Resizing the frame to reduce data size
    data = pickle.dumps(frame, protocol=pickle.HIGHEST_PROTOCOL)
    message_size = struct.pack("Q", len(data))

```

- `ret, frame = cap.read()`: Membaca frame dari objek video capture (`cap`) dan menyimpannya dalam variabel `frame`. Variabel `ret` akan menjadi `True` jika bacaan berhasil dilakukan.
- `frame = cv2.resize(frame, (640, 480))`: Mengubah ukuran frame menjadi 640x480 piksel. Ini dilakukan untuk mengurangi ukuran data yang akan dikirim.
- `data = pickle.dumps(frame, protocol=pickle.HIGHEST_PROTOCOL)`: Mengonversi frame menjadi byte-string menggunakan modul `pickle`. Ini dilakukan untuk mengirimkan frame melalui koneksi socket.
- `message_size = struct.pack("Q", len(data))`: Menghitung panjang data yang akan dikirim dan mem-packing panjang tersebut ke dalam sebuah byte-string menggunakan modul `struct`. Ini membantu dalam menentukan ukuran pesan yang akan dikirimkan melalui socket.

```

client_socket.sendall(message_size + data)

```

- `message_size`: Merupakan panjang data yang dikirimkan dalam bentuk byte-string yang sudah dipacking sebelumnya menggunakan `struct.pack`.
- `data`: Merupakan frame yang telah diubah menjadi byte-string menggunakan modul `pickle.dumps`.

```

data = receive_data(client_socket, struct.calcsize("Q"))
if not data:
    break
msg_size = struct.unpack("Q", data)[0]

```

- `receive_data(client_socket, struct.calcsize("Q"))`: Menerima data dari server dengan panjang yang sudah dikalkulasi sebelumnya menggunakan `struct.calcsize("Q")`, yang merupakan panjang byte-string yang merepresentasikan ukuran pesan.
- `if not data: break`: Jika tidak ada data yang diterima (karena koneksi terputus), loop dihentikan.
- `msg_size = struct.unpack("Q", data)[0]`: Data yang diterima di-depak menggunakan `struct.unpack("Q", data)` untuk mendapatkan panjang sebenarnya dari pesan yang akan diterima. Ini merupakan panjang pesan yang akan diterima selanjutnya.

```
frame_data = receive_data(client_socket, msg_size)
frame = pickle.loads(frame_data)
```

- `receive_data(client_socket, msg_size)`: Menerima data dari server dengan panjang yang telah ditentukan sebelumnya (`msg_size`), yang merepresentasikan panjang frame yang dikirimkan.
- `frame = pickle.loads(frame_data)`: Data frame yang diterima diubah kembali menjadi frame aslinya menggunakan `pickle.loads`, sehingga frame dapat diproses atau ditampilkan sesuai kebutuhan.

```
cv2.imshow('video', frame)
```

- Menampilkan frame video dalam jendela dengan judul 'video'. `frame` adalah variabel yang berisi frame video yang ingin ditampilkan.

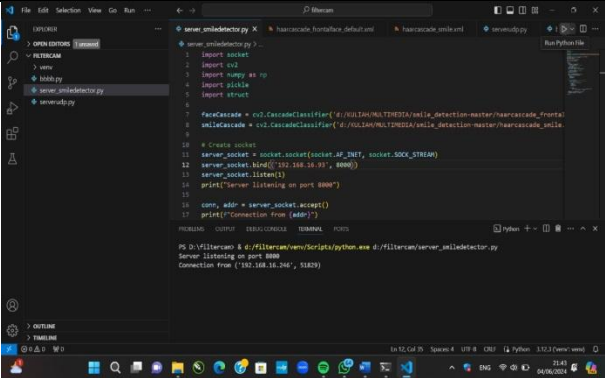
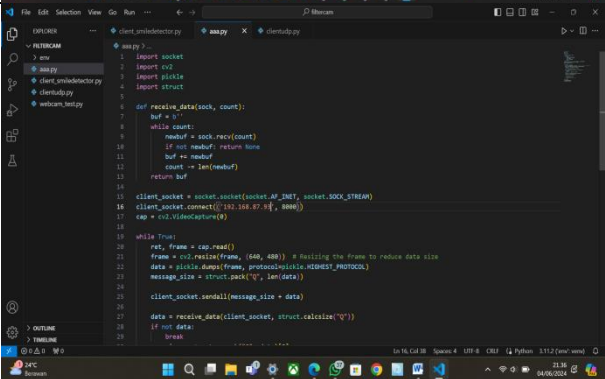
```
k = cv2.waitKey(30) & 0xff
if k == 27: # press 'ESC' to quit
    break
```

- `cv2.waitKey(30)`: Menunggu input dari keyboard selama 30 milidetik. Nilai 30 menunjukkan waktu tunggu dalam milidetik.
- `& 0xff`: Masking operasi yang digunakan untuk memastikan bahwa hanya 8 bit terakhir dari nilai yang dihasilkan yang digunakan. Hal ini memastikan kompatibilitas dengan berbagai sistem operasi.
- `if k == 27: # press 'ESC' to quit`: Memeriksa apakah nilai `k` (kode tombol yang ditekan) sama dengan 27, yang merupakan kode untuk tombol 'ESC' di keyboard.
- `break`: Jika tombol 'ESC' ditekan, loop dihentikan dan program berhenti.

```
cap.release()
cv2.destroyAllWindows()
client_socket.close()
```

- `cap.release()`: Melepaskan sumber daya kamera yang digunakan. Ini akan menghentikan akses ke kamera dan membebaskan sumber daya yang digunakan oleh objek `cap`.
- `cv2.destroyAllWindows()`: Menutup semua jendela yang telah dibuat oleh OpenCV. Ini memastikan bahwa semua jendela ditutup setelah aplikasi selesai dieksekusi.
- `client_socket.close()`: Menutup koneksi socket dengan server. Ini memastikan bahwa sumber daya jaringan yang digunakan oleh koneksi socket dilepaskan dan tidak lagi digunakan oleh aplikasi.
-

Hasil Praktikum

No	Praktikum	Hasil Screenshot
1	Server Smiledetector	 <pre> 1 import cv2 2 import socket 3 import numpy as np 4 import struct 5 6 # FaceCascade 7 faceCascade = cv2.CascadeClassifier('D:/KULIAH/ML/THESIS/face_detection-master/haarcascade_frontalface_default.xml') 8 smileCascade = cv2.CascadeClassifier('D:/KULIAH/ML/THESIS/face_detection-master/haarcascade_smile.xml') 9 10 # Create socket 11 server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM) 12 server_socket.bind(('192.168.18.204', 8080)) 13 server_socket.listen() 14 print("Server listening on port 8080") 15 16 conn, addr = server_socket.accept() 17 print("Connection from", addr) 18 19 # Receive data from client 20 data = conn.recv(1024) 21 count = 0 22 while True: 23 buf = b'' 24 while count: 25 readBuf = conn.recv(count) 26 if not readBuf: 27 return None 28 buf += readBuf 29 count -= len(readBuf) 30 return buf 31 32 # Client socket 33 client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM) 34 client_socket.connect(('192.168.18.204', 8080)) 35 cap = cv2.VideoCapture(0) 36 37 while True: 38 ret, frame = cap.read() 39 frame = cv2.resize(frame, (640, 480)) 40 data = struct.pack('f', cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY).tobytes()) 41 message_size = struct.pack('i', len(data)) 42 client_socket.sendall(message_size + data) 43 data = receive_data(client_socket, struct.calcsize('i')) 44 if not data: 45 break </pre>
2	Client Smiledetector	 <pre> 1 import socket 2 import cv2 3 import struct 4 5 # Create socket 6 client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM) 7 client_socket.connect(('192.168.18.204', 8080)) 8 9 # Receive data from server 10 data = client_socket.recv(1024) 11 count = 0 12 while True: 13 buf = b'' 14 while count: 15 readBuf = client_socket.recv(count) 16 if not readBuf: 17 return None 18 buf += readBuf 19 count -= len(readBuf) 20 return buf 21 22 # Send data to server 23 frame = cv2.imread('D:/KULIAH/ML/THESIS/face_detection-master/haarcascade_frontalface_default.xml') 24 frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) 25 message_size = struct.pack('i', len(frame.tobytes())) 26 client_socket.sendall(message_size + frame.tobytes()) 27 28 # Receive data from server 29 data = receive_data(client_socket, struct.calcsize('i')) 30 if not data: 31 break </pre>
3	Hasil praktikum	